

1. Archivos en Java. Clase File.

Uno de los principales problemas que hemos encontrado en nuestro aprendizaje del desarrollo de software con Java, es que todos los programas que hemos realizado hasta el momento pierden su información al terminar de ejecutarse.

La gestión de datos e información por parte de los programas informáticos es un componente esencial para poder desarrollar aplicaciones versátiles. En la actualidad existen diferentes métodos de almacenar y obtener datos, algunos de estos son ficheros, bases de datos, consultas a APIs, etc. En esta unidad vamos a entender cómo trabaja Java con los ficheros y algunas de sus utilidades.

Java tiene dos librerías principales para trabajar con ficheros:

- `java.io`

Esta fue la primera librería que se desarrolló y una de las más empleadas.

- `java.nio`

Esta librería se creó para solucionar algunas de las limitaciones de `java.io`, por ejemplo, `java.nio` tiene mejores utilidades para realizar operaciones de entrada y salida de alta velocidad (New I/O).

En esta unidad de trabajo nos centraremos en `java.nio`. Aunque muchas aplicaciones aún usan `java.io`, `java.nio` ofrece ventajas significativas en rendimiento y flexibilidad, especialmente en aplicaciones modernas.

Los ficheros son secuencias de datos almacenados en un dispositivo de almacenamiento, existen diferentes sistemas de gestión de ficheros, pero estos quedan fuera del alcance de este módulo. Java permite leer y escribir de ficheros de texto (secuencias de caracteres) o ficheros binarios (secuencia de bytes que pueden tener cualquier tipo de información: imágenes, audio, etc.).

La clase `Path`

Proporciona múltiples herramientas para trabajar con archivos y directorios. Esta clase no lee ni escribe directamente en los archivos; estas operaciones las realizan otras clases que estudiaremos en las próximas lecciones, pero cualquier operación con directorios o ficheros necesitará usar instancias de objetos `Path`.

La clase Path contiene información sobre ficheros y directorios que podremos usar de múltiples formas mediante sus métodos. Explora la información de la API y el código proporcionado para familiarizarte con las herramientas que ofrece.

Ejemplo de instancia de Path:

```
Path fichero = Paths.get("miFichero.txt");
```

Paths (Clase utilitaria): fábrica de objetos Path

La clase Paths contiene métodos estáticos para crear instancias de Path.

Ejemplo de uso:

```
Path ruta1 = Paths.get("archivo.txt"); // Ruta relativa
```

```
Path ruta2 = Paths.get("C:", "Users", "doc.txt"); // Ruta absoluta
```

Files (Clase utilitaria): operaciones con archivos

La clase Files proporciona todos los métodos para operaciones reales con archivos:

- Creación/eliminación de archivos y directorios
- Lectura/escritura de contenido
- Copiar/mover archivos
- Consulta de atributos

Ejemplo de uso:

```
// Ejemplo combinando las tres clases
Path ruta = Paths.get("datos.txt");
if (Files.notExists(ruta)) {
    Files.createFile(ruta);
}
```

Cuando trabajamos con ficheros es importante entender las diferentes excepciones que pueden ocurrir en nuestro programa. ¿Qué pasa si intento escribir en un archivo para el que no tengo permisos de escritura? ¿Y si intento leer un archivo que no existe? El sistema de ficheros es algo externo al programa, es por ello que debemos entender todas las operaciones que hagamos con ficheros, investigar si es posible que éstas generen excepciones y capturarlas y manejarlas de la forma deseada.

Ejercicios

Ejercicio 1.

Muestra todos los archivos de una carpeta junto con su tamaño. Métodos relevantes:

`Files.isDirectory(Path)`, `Files.list(Path)`, `Files.isRegularFile(Path)`, `Path.getFileName()`,
`Files.size(Path)` (en bytes).

Ejercicio 2.

Lista todos los archivos de un árbol de directorios. Tiene que entrar en tantas carpetas como tenga el árbol (similar a lo que hace el comando `tree` en linux).

Ejercicio 3.

Explora las clases `Path`, `Files` y `Paths` en el API de Java NIO e identifica diferentes excepciones generadas por algunos de sus métodos principales.